FastVGGT: Training-Free Token Merging for Long-Sequence Visual Geometry Transformers

Anonymous Author

Abstract

Visual Geometry Grounded Transformer (VGGT) [12] is a strong feed-forward model for joint 3D geometry and camera estimation, but its global attention becomes increasingly expensive on long image sequences and can amplify small errors across many frames. We show that VGGT's global attention maps exhibit strong cross-token similarity, indicating substantial redundancy that can be removed at inference time. Based on this observation, we propose FastVGGT, a training-free acceleration method that inserts geometry-aware token merging into VGGT's global attention blocks. FastVGGT preserves all tokens from the first frame as geometric anchors, keeps per-frame special tokens and a small fixed-stride subset of image tokens to maintain correspondences, samples additional destination tokens uniformly over the image grid, merges redundant source tokens to their nearest destinations using cosine similarity, and restores the original token resolution before dense prediction. We further use a memory-cleaned VGGT inference path to evaluate thousand-frame inputs without changing model weights. On ScanNet-50, FastVGGT reduces reconstruction time from 724.6s to 180.7s on 1000-frame sequences while improving Chamfer distance from 0.471 to 0.425; it also improves pose estimation on the same benchmark, reducing ATE from 0.196 to 0.164 and ARE from 4.636 to 3.860. On 7 Scenes and NRGBD, FastVGGT preserves or slightly improves reconstruction quality while cutting runtime by 2.6×–2.7×. These results show that training-free, 3D-aware token merging is an effective way to scale feed-forward geometry transformers to long image sequences.*

1. Introduction

Feed-forward 3D vision models have recently narrowed the gap between geometric pipelines and end-to-end transformers. In particular, VGGT [12] predicts cameras, point maps, depth, and tracks from a set of images in a single forward pass, avoiding iterative multi-view optimization while achieving strong performance. This design is attractive for long-sequence reconstruction because it reasons jointly across the full image set. However, the same property becomes a bottleneck when the input grows to hundreds or thousands of frames: global attention scales quadratically with token count, memory use rises sharply, and small geometric errors can accumulate across the dense all-to-all interactions.

Existing solutions for long-range 3D perception usually change the model structure. CUT3R [13] and StreamVGGT [18] process images incrementally, while VGGT-Long [6] uses chunking and alignment. These are effective, but they are not drop-in accelerations of pretrained feed-forward VGGT. Our goal is to accelerate long-sequence inference without retraining.

Our starting observation is that many VGGT attention maps look strikingly similar across blocks, suggesting nearly redundant global interactions. This creates an opportunity for training-free token merging, but naive merging is unsafe in 3D because geometric consistency depends on stable anchors, broad spatial coverage, and dense-output support.

We therefore propose **FastVGGT**, a training-free token-merging strategy specialized for VGGT. FastVGGT preserves the entire first frame as a world anchor, preserves special tokens and a small fixed-stride subset of tokens that stabilize correspondences, and samples destination tokens from spatial regions to maintain uniform coverage. Redundant source tokens are merged to nearby destination tokens only within the global-attention module, and an unmerging step restores the original token layout before downstream dense prediction. The method requires no fine-tuning and leaves the pretrained weights unchanged.

The results support this design. On ScanNet-50 with 1000 images, FastVGGT is 4.0× faster than VGGT* while improving Chamfer distance from 0.471 to 0.425. The same trend appears in pose estimation, where ATE drops from 0.196 to 0.164 and RPE-rot from 0.997 to 0.667. On 7 Scenes and NRGBD, FastVGGT keeps reconstruction quality nearly unchanged while reducing runtime by 2.6×–2.7×.

Our contributions are threefold:

- We identify global-attention redundancy in VGGT through attention visualization and timing analysis, and

AI-Generated

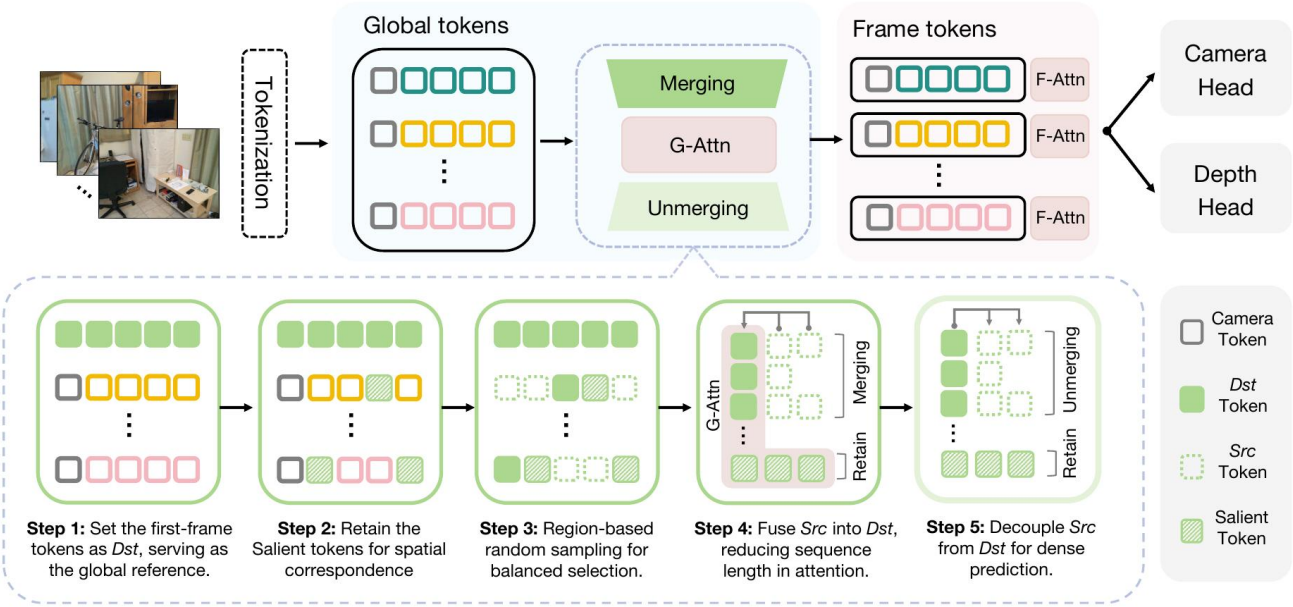

Figure 1. Overview of FastVGGT. We partition tokens into protected destination tokens and mergeable source tokens inside global attention only. All first-frame tokens are kept as anchors, special tokens are preserved, and additional destination tokens are sampled uniformly over later frames. Source tokens are merged to their nearest destination tokens during attention and are restored to the original layout before dense prediction heads.

show that this redundancy is a practical bottleneck for long-sequence inference.

- We introduce a training-free, 3D-aware token-merging method for feed-forward geometry transformers that preserves anchors and dense prediction compatibility through structured partitioning and unmerging.
- We demonstrate up to 4.0× faster thousand-frame inference with equal or better reconstruction and pose accuracy, with the largest gains appearing on the longest sequences.

2. Related Work

2.1. Feed-forward and recurrent 3D reconstruction

Recent 3D vision models increasingly replace optimization-heavy pipelines with feed-forward inference. DUSt3R [14] predicts point maps, Fast3R [16] extends this idea to many-view reconstruction, and VGGT [12] unifies multiple geometric outputs within one transformer. In parallel, recurrent or streaming approaches such as CUT3R [13], StreamVGGT [18], and VGGT-Long [6] improve scalability by changing the inference pattern. FastVGGT is complementary: we directly accelerate pretrained feed-forward VGGT.

2.2. Token reduction in transformers

Token pruning and merging are widely used to accelerate transformers. ToMe [2] introduced training-free token merging for vision transformers, and later work extended merging to diffusion models [1], video transformers [5, 8, 11], vision-language models [4], and hybrid pruning-merging schemes [3]. Other work studies which tokens are safe to remove or preserve [7], or learns token aggregation directly [9, 10, 17]. These methods show that transformer representations contain redundancy, but they are mostly designed for recognition, generation, or video understanding. Our setting is stricter: dense 3D geometry requires maintaining correspondence structure, stable reference frames, and per-pixel outputs. FastVGGT adapts training-free token merging to this regime with geometry-aware token partitioning and explicit unmerging.

2.3. Why generic token merging is insufficient for 3D geometry

Generic token reduction methods usually optimize semantic accuracy or throughput, not geometric consistency. In long-sequence 3D reconstruction, removing the wrong tokens can corrupt camera alignment, destabilize the common coordinate frame, and break dense prediction heads. Our ablations show precisely this failure mode: naive random or fixed-stride merging degrades reconstruction quality substantially, despite reducing runtime. FastVGGT addresses

AI-Generated

the gap by separating anchor preservation, uniform spatial coverage, and protected correspondence tokens within the merging design.

3. Method

3.1. Background and motivation

VGGT alternates frame-local and global-attention blocks over tokens extracted from all input images [12]. Let a sequence contain S frames, each represented by P patch tokens and 5 special tokens, giving $N = S(P + 5)$ tokens in total. The global-attention blocks dominate long-sequence cost because they perform dense attention over all N tokens; their cost and memory both grow quadratically with sequence length.

Figure 3 provides the empirical motivation for our method. The timing breakdown shows that global attention is the dominant inference component on long inputs, while attention-map visualization reveals that different query tokens often attend to nearly the same spatial regions. This redundancy suggests that many tokens can share global context with limited loss, as long as geometrically important tokens are protected.

3.2. 3D-aware token partitioning

FastVGGT modifies only global-attention blocks. For each block, we partition tokens into *destination* tokens, which remain explicit during attention, and *source* tokens, which may be merged into destinations.

Reference anchors. All tokens from the first frame are always kept as destination tokens. This includes both patch tokens and special tokens. The first frame acts as a stable reference coordinate system, and preserving it prevents the common frame from drifting when later-frame tokens are compressed.

Special and spatially distributed destination tokens. For every later frame, the 5 special tokens are also preserved. Among patch tokens, we divide the 2D token grid into non-overlapping 2×2 regions and sample one destination token per region, yielding spatially uniform coverage over the image. The remaining tokens in those regions become candidate source tokens. This region-wise sampling is implemented with a fixed random seed for deterministic evaluation.

Protected correspondence tokens. In addition to the destination set above, we protect roughly 10% of token positions using a fixed-stride deterministic mask. These tokens are excluded from merging and passed through unchanged. Although simple, this protection mechanism preserves a sparse set of consistent token locations across frames and improves both reconstruction and pose estimation in our ablations.

3.3. Training-free merging and unmerging

Let $X \in \mathbb{R}^{N \times C}$ denote the input token features of a global-attention block. FastVGGT computes the merge assignment from the normalized token embeddings themselves, before attention. For each source token, we find its nearest destination token under cosine similarity. We then rank source tokens by their maximum destination similarity and merge the top-r source tokens, where r is controlled by a merge ratio hyperparameter.

The same merge assignment is applied to the query, key, and value tensors. Each selected source token is merged into its matched destination token with mean aggregation. Source tokens that are not selected for merging remain explicit. This produces a shorter token sequence for the expensive global-attention computation.

Because VGGT predicts dense geometry, reducing the token count permanently would break downstream heads. FastVGGT therefore performs an *unmerge* immediately after the global-attention output projection: each original source position is restored by copying back the feature of its matched destination token, while unmerged and protected tokens return to their original positions unchanged. In this way, every later block and prediction head receives tokens in the original layout.

3.4. Effective sequence length and compute

If N_{dst} and N_{src} denote the destination and candidate source counts, and $\rho \in [0, 1]$ is the merge ratio, then the global-attention block operates on

$$N' = N_{\text{dst}} + (1 - \rho)N_{\text{src}}$$

tokens instead of $N = N_{\text{dst}} + N_{\text{src}}$. The dominant attention cost therefore changes from $\mathcal{O}(N^2C)$ to $\mathcal{O}(N'^2C)$, while the unmerge step restores the original length N before the next block. This separation is important: FastVGGT saves compute exactly where VGGT is most expensive, but leaves the downstream dense heads and pretrained parameters unchanged.

3.5. Where merging is applied

FastVGGT applies merging only in global-attention blocks, not in frame-local attention. This choice matches the runtime profile in Figure 3: global attention is the dominant cost, while frame-local attention is much cheaper and contains within-frame detail that we prefer not to compress. The implementation supports starting from any global-attention block index; our ablations show that merging from the earliest global-attention block is both faster and more accurate than delaying merging to later blocks.

AI-Generated

3.6. Memory-cleaned VGGT* baseline

Long sequences can exceed memory even without token merging. To make the comparison fair and to enable 1000-frame evaluation, we use a memory-cleaned inference path denoted VGGT*. This variant does not change the pre-trained model weights or prediction heads; it only discards unused intermediate tensors and aggressively clears caches during inference. FastVGGT is built on top of the same inference path, so the comparison against VGGT* isolates the effect of token merging.

4. Experiments

4.1. Experimental setup

We evaluate on three benchmarks provided with the project code: ScanNet-50, 7 Scenes, and NRGBD. For ScanNet-50, we test 1000, 500, 300, and 100 input images. For 7 Scenes and NRGBD, we evaluate keyframe sampling at strides 3 and 10. Reconstruction quality is measured by Chamfer distance (CD) on ScanNet-50, and by accuracy (Acc), completeness (Comp), and normal consistency (NC) on 7 Scenes and NRGBD, following the released evaluation scripts. We additionally report camera pose metrics on ScanNet-50: absolute trajectory error (ATE), absolute rotation error (ARE), relative pose error for rotation (RPE-rot), and relative pose error for translation (RPE-trans).

We compare against π^3 [15], StreamVGGT [18], Fast3R [16], CUT3R [13], and the memory-cleaned VGGT* baseline. Unless stated otherwise, FastVGGT uses the strongest ablation setting from Table 7: merging begins from the first global-attention block and uses merge ratio 0.9.

4.2. Reconstruction on ScanNet-50

Table 1 shows that FastVGGT is most beneficial on long sequences. On 1000 frames, it improves CD from 0.471 to 0.425 while reducing runtime from 724.6s to 180.7s, a 4.0× speedup. On 500 frames, it improves CD from 0.420 to 0.411 and reduces runtime from 177.5s to 55.2s, a 3.2× speedup. On 300 frames, FastVGGT matches VGGT* in CD (0.416) while running 5.5× faster. On 100 frames, the speedup remains useful (9.1s to 5.4s) but CD is slightly worse (0.426 vs. 0.423), indicating that the main gains come from mitigating redundancy and error accumulation in longer sequences rather than short ones.

Compared with other scalable baselines, FastVGGT obtains the best reconstruction quality at 500 and 1000 frames. Fast3R and CUT3R are faster than the uncompressed VGGT* on some shorter settings, but their CD is markedly worse. π^3 and StreamVGGT run out of memory in this evaluation.

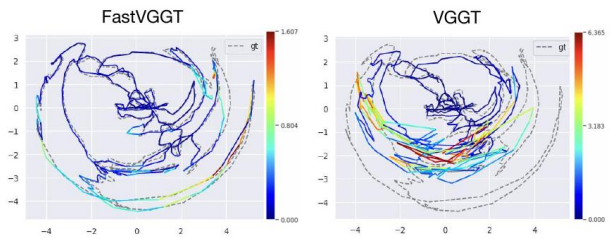

Figure 2. Qualitative pose comparison on ScanNet-50. FastVGGT better preserves long-range trajectory consistency than the VGGT* baseline on the longest sequences, matching the quantitative gains in Table 4.

4.3. Reconstruction on 7 Scenes and NRGBD

Tables 2 and 3 show that FastVGGT transfers to indoor benchmarks beyond ScanNet-50. On 7 Scenes, FastVGGT improves or matches VGGT* on nearly every metric while reducing runtime from 76.7s to 28.0s at stride 3 and from 8.7s to 5.1s at stride 10. The gains are especially clear on the denser stride-3 setting, where FastVGGT improves Acc from 0.019 to 0.018 and NC from 0.611 to 0.617.

On NRGBD, the picture is more mixed but still favorable overall. At stride 3, FastVGGT matches mean Acc (0.029), slightly worsens completeness (0.019 vs. 0.018), improves NC (0.730 vs. 0.727), and reduces runtime from 136.1s to 53.1s. At stride 10, distance metrics are slightly worse than VGGT*, but the changes are small and runtime is nearly halved (13.9s to 7.3s). These results again suggest that FastVGGT is most helpful when the global context is long and redundant.

4.4. Camera pose estimation on ScanNet-50

FastVGGT also benefits pose estimation. Table 4 shows consistent improvements over VGGT* at 500 and 1000 frames: at 1000 frames, ATE improves from 0.196 to 0.164, ARE from 4.636 to 3.860, RPE-rot from 0.997 to 0.667, and RPE-trans from 0.039 to 0.029. At 300 and 100 frames, the results are mixed. FastVGGT still improves ARE, but ATE and some relative pose errors are slightly worse than the baseline. This mirrors the reconstruction results and supports the claim that FastVGGT mainly addresses the failure mode of very long sequences, where dense global attention becomes both expensive and less stable.

5. Analysis

5.1. Why geometry-aware partitioning matters

Table 5 shows that directly importing generic merging heuristics is not sufficient. Random sampling and fixed-stride sampling reduce runtime relative to the baseline, but both increase CD substantially; for example, fixed-stride sampling with stride 8 reduces runtime from 30.4 to 11.6

AI-Generated

Method	1000		500		300		100	
	CD ↓	Time ↓	CD ↓	Time ↓	CD ↓	Time ↓	CD ↓	Time ↓
π^3 [15]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
StreamVGGT[18]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
Fast3R [16]	0.684	397.8s	0.701	97.3s	0.711	34.9s	0.723	4.8s
CUT3R [13]	0.786	34.8s	0.774	18.8s	0.775	11.1s	0.767	3.6s
VGGT* [12]	0.471	724.6s	0.420	177.5s	**0.416**	131.4s	**0.423**	9.1s
FastVGGT	**0.425**	**180.7s**	**0.411**	**55.2s**	**0.416**	**23.8s**	0.426	**5.4s**

Table 1. Quantitative results of point cloud reconstruction on the ScanNet-50 dataset with input sequences of 1000, 500, 300, and 100 images. *OOM* denotes out-of-memory.

Method	7 Scenes - Stride 3							7 Scenes - Stride 10						
	Acc ↓		Comp ↓		NC ↑		Time ↓	Acc ↓		Comp ↓		NC ↑		Time ↓
	Mean	Med.	Mean	Med.	Mean	Med.		Mean	Med.	Mean	Med.	Mean	Med.	
π^3 [15]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
StreamVGGT [18]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
Fast3R [16]	0.045	0.027	0.047	0.010	0.616	0.627	43.7s	0.040	0.021	0.056	0.013	0.639	0.657	5.5s
CUT3R [13]	0.179	0.121	0.097	0.043	0.588	0.581	14.5s	0.041	0.021	0.029	0.010	0.651	0.677	4.2s
VGGT* [12]	0.019	0.008	0.027	0.010	0.611	0.628	76.7s	0.020	0.008	0.027	0.010	0.623	0.641	8.7s
FastVGGT	**0.018**	**0.008**	**0.026**	**0.010**	**0.617**	**0.634**	**28.0s**	**0.018**	**0.008**	**0.027**	**0.010**	**0.628**	**0.648**	**5.1s**

Table 2. Quantitative results of point cloud reconstruction on the 7 Scenes dataset. Stride denotes keyframes sampled every 3 or 10 frames.

AI-Generated

while worsening CD from 0.21 to 0.77. This confirms that speed alone is not the right objective in long-sequence 3D reconstruction.

Table 6 breaks the FastVGGT partition into three ingredients: uniform spatial coverage, reference anchors, and protected fixed-stride tokens. Uniform coverage alone is not enough, yielding CD 0.637 and ATE 0.722. Adding the reference frame dramatically improves both metrics (CD 0.431, ATE 0.149), showing that stable anchors are essential. Adding protected fixed-stride tokens further improves the result to CD 0.411 and ATE 0.141, indicating that sparse correspondence preservation remains useful even after anchor preservation.

5.2. Effect of merge ratio and merge depth

Table 7 studies the merge ratio and the first block where merging is enabled. The best overall trade-off is obtained by starting from block 0 and using ratio 0.9, which gives CD 0.411 in 55.2s. Delaying merging to block 10 or 20 consistently increases runtime and usually degrades reconstruction quality. This result is intuitive: if redundancy already exists in early global-attention blocks, exploiting it late wastes computation without preserving useful detail.

5.3. Discussion

Two trends are consistent across all experiments. First, FastVGGT helps most on the longest inputs, where global-attention redundancy and error accumulation are largest. Second, the method is robust because it compresses only the expensive global-attention computation while restoring the original token layout before dense prediction. Figure 4 summarizes this trade-off: the largest runtime gains appear exactly in the regime where keeping all tokens active is least efficient. These two properties explain why FastVGGT can improve accuracy at 500–1000 frames, yet remain close to the baseline even when the sequence is short enough that compression is less necessary.

6. Conclusion

We presented FastVGGT, a training-free acceleration method for long-sequence VGGT inference. The central idea is that global attention in VGGT contains substantial token redundancy, but exploiting that redundancy for 3D geometry requires more structure than generic token merging. By preserving the first frame as a reference anchor, keeping special and protected correspondence tokens, sampling destination tokens uniformly, and unmerging before dense

Method	NRGBD - Stride 3							NRGBD - Stride 10						
	Acc ↓		Comp ↓		NC ↑		Time ↓	Acc ↓		Comp ↓		NC ↑		Time ↓
	Mean	Med.	Mean	Med.	Mean	Med.		Mean	Med.	Mean	Med.	Mean	Med.	
π^3 [15]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
StreamVGGT [18]	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*	*OOM*
Fast3R [16]	0.074	0.036	0.024	0.011	0.658	0.682	68.9s	0.061	0.028	0.031	0.013	0.669	0.712	7.4s
CUT3R [13]	0.346	0.243	0.184	0.090	0.579	0.623	18.3s	0.132	0.064	0.056	0.0106	0.669	0.725	5.7s
VGGT* [12]	**0.029**	**0.019**	**0.018**	**0.009**	0.727	0.728	136.1s	**0.016**	**0.010**	**0.017**	**0.009**	0.735	0.738	13.9s
FastVGGT	**0.029**	0.021	0.019	0.010	**0.730**	**0.738**	**53.1s**	0.018	0.011	0.018	**0.009**	**0.736**	**0.741**	**7.3s**

Table 3. Quantitative results of point cloud reconstruction on the NRGBD dataset.

Input	ATE ↓		ARE ↓		RPE-rot ↓		RPE-trans ↓	
Frames	Baseline	Ours	Baseline	Ours	Baseline	Ours	Baseline	Ours
1000	0.196	**0.164**	4.636	**3.860**	0.997	**0.667**	0.039	**0.029**
500	0.174	**0.145**	4.190	**3.591**	0.963	**0.627**	0.042	**0.031**
300	0.145	**0.142**	3.689	**3.554**	**0.786**	0.801	0.040	**0.036**
100	**0.140**	0.141	3.625	**3.512**	**1.224**	1.262	**0.058**	0.061

Table 4. Quantitative results of camera pose estimation on the ScanNet-50 dataset.

	Random		Fixed		Baseline
	$r = 0.5$	$r = 0.8$	$s = 5$	$s = 8$	
CD ↓	0.32	0.44	0.33	0.77	0.21
Time ↓	22.6	16.9	17.2	11.6	30.4

Table 5. Results of applying merging strategies based on random sampling and fixed-stride sampling. Here, r denotes the merging ratio, and s represents the stride for selecting dst tokens.

Methods	Uniform	Reference	Salient	CD ↓	ATE ↓
(a)	-	-	-	0.947	0.842
(b)	✓	-	-	0.637	0.722
(c)	✓	✓	-	0.431	0.149
(d)	✓	✓	✓	0.411	0.141

Table 6. Ablation study for Token partitioning strategies.

Blocks	0.3		0.6		0.9	
	CD ↓	Time ↓	CD ↓	Time ↓	CD ↓	Time ↓
0	0.408	119.8s	0.415	64.3s	0.411	55.2s
10	0.418	146.2s	0.424	118.4s	0.431	106.3s
20	0.423	172.9s	0.427	169.1s	0.411	157.3s

Table 7. Ablation study on merging ratio and affected blocks.

prediction, FastVGGT achieves large runtime reductions without retraining and without sacrificing geometric quality. Across ScanNet-50, 7 Scenes, and NRGBD, FastVGGT delivers up to 4.0× faster inference and is strongest exactly where long-sequence processing is hardest: thousand-frame reconstruction and pose estimation.

AI-Generated

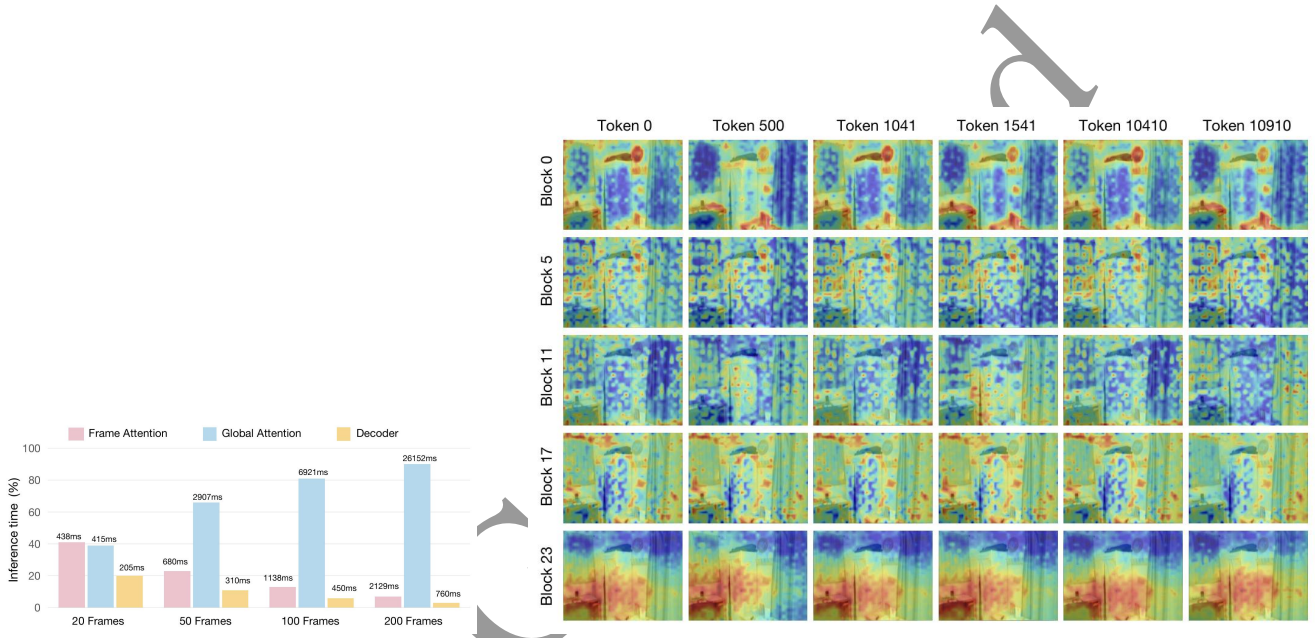

Figure 3. Motivation for FastVGGT. Left: component-wise inference time of VGGT shows that global attention dominates long-sequence runtime. Right: attention maps for several representative tokens are highly similar across stages, revealing strong redundancy in global interactions.

AI-Generated

Figure 4. Summary of the FastVGGT design and its long-sequence effect. FastVGGT shortens the sequence only inside global attention, then restores the full token layout before prediction. On ScanNet-50, this produces a much lower inference-time curve than VGGT and VGGT* as the number of frames grows, and the qualitative 1000-frame example shows cleaner reconstruction than the uncompressed baseline.

References

[1] Daniel Bolya and Judy Hoffman. Token merging for fast stable diffusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4599–4603, 2023.

[2] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your vit but faster. *arXiv preprint arXiv:2210.09461*, 2022.

[3] Maxim Bonnaerens and Joni Dambre. Learned thresholds token merging and pruning for vision transformers. *arXiv preprint arXiv:2307.10780*, 2023.

[4] Qingqing Cao, Bhargavi Paranjape, and Hannaneh Hajishirzi. Pumer: Pruning and merging tokens for efficient vision language models. *arXiv preprint arXiv:2305.17530*, 2023.

[5] Joonmyung Choi, Sanghyeok Lee, Jaewon Chu, Minhyuk Choi, and Hyunwoo J Kim. vid-tldr: Training free token merging for light-weight video transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18771–18781, 2024.

[6] Kai Deng, Zexin Ti, Jiawei Xu, Jian Yang, and Jin Xie. Vggt-long: Chunk it, loop it, align it–pushing vggt's limits on kilometer-scale long rgb sequences. *arXiv preprint arXiv:2507.16443*, 2025.

[7] Joakim Bruslund Haurum, Sergio Escalera, Graham W Taylor, and Thomas B Moeslund. Which tokens to use? investigating token reduction in vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 773–783, 2023.

[8] Seon-Ho Lee, Jue Wang, Zhikang Zhang, David Fan, and Xinyu Li. Video token merging for long-form video understanding. *arXiv preprint arXiv:2410.23782*, 2024.

[9] Cedric Renggli, André Susano Pinto, Neil Houlsby, Basil Mustafa, Joan Puigcerver, and Carlos Riquelme. Learning to merge tokens in vision transformers. *arXiv preprint arXiv:2202.12015*, 2022.

[10] Michael Ryoo, AJ Piergiovanni, Anurag Arnab, Mostafa Dehghani, and Anelia Angelova. Tokenlearner: Adaptive space-time tokenization for videos. *Advances in neural information processing systems*, 34:12786–12797, 2021.

[11] Leqi Shen, Tianxiang Hao, Tao He, Sicheng Zhao, Yifeng Zhang, Pengzhang Liu, Yongjun Bao, and Guiguang Ding. Tempme: Video temporal token merging for efficient text-video retrieval. *arXiv preprint arXiv:2409.01156*, 2024.

[12] Jianyuan Wang, Minghao Chen, Nikita Karaev, Andrea Vedaldi, Christian Rupprecht, and David Novotny. Vggt: Visual geometry grounded transformer. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 5294–5306, 2025.

[13] Qianqian Wang, Yifei Zhang, Aleksander Holynski, Alexei A Efros, and Angjoo Kanazawa. Continuous 3d perception model with persistent state. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 10510–10522, 2025.

[14] Shuzhe Wang, Vincent Leroy, Yohann Cabon, Boris Chidlovskii, and Jerome Revaud. Dust3r: Geometric 3d vision made easy. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20697–20709, 2024.

[15] Yifan Wang, Jianjun Zhou, Haoyi Zhu, Wenzheng Chang, Yang Zhou, Zizun Li, Junyi Chen, Jiangmiao Pang, Chunhua Shen, and Tong He. Scalable permutation-equivariant visual geometry learning. *arXiv preprint arXiv:2507.13347*, 2025.

[16] Jianing Yang, Alexander Sax, Kevin J Liang, Mikael Henaff, Hao Tang, Ang Cao, Joyce Chai, Franziska Meier, and Matt Feiszli. Fast3r: Towards 3d reconstruction of 1000+ images in one forward pass. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 21924–21935, 2025.

[17] Wang Zeng, Sheng Jin, Wentao Liu, Chen Qian, Ping Luo, Wanli Ouyang, and Xiaogang Wang. Not all tokens are equal: Human-centric visual analysis via token clustering transformer. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11101–11111, 2022.

AI-Generated

[18] Dong Zhuo, Wenzhao Zheng, Jiahe Guo, Yuqi Wu, Jie Zhou, and Jiwen Lu. Streaming 4d visual geometry transformer. *arXiv preprint arXiv:2507.11539*, 2025.

AI-Generated